

and IntelWiDiExtensions_i.c. Add both of these files to your project.

- 6 Right-click the IntelWiDiExtensions_i.c file in Solution Explorer and choose Properties. In the C/C++ -> Precompiled Headers section, set Precompiled Header to "Not Using Precompiled Headers".

- 7 Now, to initialize a smart pointer that references the WiDiExtensions COM object, you can use the following lines of code:

```
// Add the following line to the top of the file
#include "IntelWiDiExtensions_h.h"
// Add the following code wherever you want to initialize your smart pointer
CComPtr<IWiDiExtensions> pWiDi;
HRESULT hr = pWiDi.CoCreateInstance(CLSID_WiDiExtensions);
// Check if smart pointer initialization was successful
if (FAILED(hr))
    // TODO: Add code to handle failure here
HRESULT hr = pWiDi->Initialize((DWORD)AfxGetMainWnd()->GetSafeHwnd());
```

- 8 Now you can call any of the Extensions SDK functions directly using the smart pointer. Other ways of accessing the COM object from C++ can also be employed here. We can, for example, use the IDL file to generate a type library (TLB) which can be imported into the project. Once this is done, the sample application can be developed further to provide WiDi functions to the application.

DEPLOYING THE WIDI APPLICATION

As mentioned before, the DLL provided by the SDK needs to be registered when the application using this SDK is installed on any machine. This DLL needs to be placed in the following directory: -

C:\Program Files (x86)\Common Files\Intel Corporation\Intel WiDi Extensions SDK

In the Windows MSI install project, the value of the Register attribute can be set to vsdrpCOMSelfReg which will cause this DLL to be registered automatically. Also, since this DLL is a shared library, the Permanent attribute needs to be set as true, so that it is not removed when the application is uninstalled.

INTEL COMPILER OPTIONS FOR HASWELL

Compiler options:

-Qxcore-avx2, -Qaxcore-avx2 (Windows*)

-xcore-avx2 and -axcore-avx2 (Linux)

-march=core-avx2 Intel compiler

Separate options for FMA:

-Qfma, -Qfma- (Windows)

-fma, -no-fma (Linux)

Visual Studio 11 and GNU Compiler 4.8 have similar support as the Intel Compiler

INTEL MEDIA SDK

The Intel Media SDK provides an Application Programming Interface to develop media applications related to video editing, video processing, media conversion, streaming, playback and other such similar applications. It is aimed at optimizing applications to run on Intel-based graphics solutions such as the Intel HD Graphics, which provides hardware acceleration for most of the common functions. In order to get started with the Media SDK, introductory tutorials can be retrieved from the Intel website that perform some basic functions such as decoding, encoding and performing video pre/post processing functions.

TOOLS FOR DEVELOPING SOFTWARE FOR HASWELL DEVICES: INTEL SOFTWARE DEVELOPMENT EMULATOR

The development tools which support software development with the Haswell New Instructions include a minimum requirement of Microsoft Visual Studio 2012 for Windows and at least GCC 4.7 on Linux. In order for the compiler to support instructions for Haswell, an assembler which is a part of the binutils package, version 2.22 or greater, is required. In order to run these programs compiled for newer platforms on older devices, the Intel Software Development Emulator can be used to emulate these programs. This emulator supports all the three platforms i.e. Windows, Linux and OS X. It is able to successfully emulate the Restricted Transactional Memory Extension that we discussed earlier. Other features include support for Intel SSE4, AES, Intel AVX, AVX2, BMI1, BMI2 and similar instructions

which were introduced in Haswell and have been discussed before.

This Software Development Emulator does a good job making the developers familiar with the upcoming instruction sets and helps them ensure that software is ready to be run on these platforms by the time they are released to the market. Apart from the emulation functions, this software also provides the following additional features: -

- **Mix Histogramming Tool:** It produces histograms related to the instructions executed.
- **Debugtrace ASCII Tracing Tool:** As the name suggests, it provides a trace of the instructions executed and information of read/write operations on the registers and memory.
- **Fingerprint Tool:** It provides a count of the number of unique 16-byte chunks of the memory which was referenced during the execution of the program.
- **XED Command line Tool:** It allows you to disassemble the PECOFF or ELF binary executable.

INTEL GRAPHICS PERFORMANCE ANALYZER

Intel GPA is one of the best tools for helping game developers to perform graphical analysis on their applications. It provides many metrics for helping the developer recognize the performance bottlenecks in their applications so that they can perform relevant optimizations. It provides the HUD view or the System Analyzer view to provide metrics data in real time for any application. Apart from this, it also has a Frame Analyzer tool to capture a frame while providing a large number of metrics to analyze data related to the frame. Graphics intensive applications targeted at Windows 7/8 and Intel-based Android devices are one of those which can benefit the most from this tool.

INTEL VTUNE AMPLIFIER XE

This tool is suggested by Intel to study CPU performance and provide instruction level analysis. It can be used to profile code and perform software sampling on x86-based devices from within Visual Studio or as an independent application. This tool is really good at providing metrics for subroutines and functions, and can be used to track down which of them is taking the maximum amount of time to execute. ■